Задача 1. Библиотека линейной алгебры

Источник:	базовая
Имя входного файла:	--
Имя выходного файла:	--
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Чтобы сдать в бот задачи из нескольких файлов, необходимо создать архив .zip.

Для выполнения вычислений над векторами и матрицами большого размера обычно используют библиотеку BLAS (Basic Linear Algebra Subprograms). Данная библиотека содержит несколько десятков функций, расфасованных по трём уровням согласно асимптотическому времени работы. Более сложные операции линейной алгебры (вроде решения систем уравнения) обычно реализуют с использованием функций BLAS. Существует множество реализаций библиотеки BLAS: проприетарные и open-source, простые и оптимизированные, для обычного процессора, видеокарты или суперкомпьютера, и так далее. Конечно, большая часть реализаций оптимизируется до предела, чтобы выжать максимальную производительность из конкретной аппаратуры.

В этой задаче предлагается написать простую реализацию некоторых функций BLAS. Сигнатуры функций настоящего BLAS очень громоздкие (например, функция перемножения матриц принимает 13 параметров), поэтому для данной задачи они несколько упрощены. Будем считать, что матрица размера m на n хранится в памяти в row-major формате без пропусков, то есть если указатель на начало матрицы равен pMatr, то элемент матрицы в i-ой строке в j-ом столбце лежит в pMatr[i * n + j]. Все векторы, матрицы и скаляры в данной задаче содержат вещественные числа типа double.

Библиотека должна реализовывать все функции из представленного на следующей странице хедера myblas.h.

На проверку требуется отправить три файла: level1.c, level2.c и level3.c. В этих файлах должны быть реализованы функции первого, второго и третьего уровня соответственно. Хедер myblas.h с указанным выше содержимым будет доступен в текущей директории.

Кроме того, вы должны научиться собирать из вашего кода статическую библиотеку. Для этого нужно дополнительно отправить два файла:

- c.bat.txt — скрипт для сборки myblas.lib компилятором MSVC.
- c.sh.txt — скрипт для сборки myblas.a компилятором GCC.

Каждый скрипт сборки должен содержать одну или несколько команд, которые нужно запускать в командной строке. Команды будут запускаться по очереди в той же директории, куда будет распакованы/сложены все ваши файлы. Разрешается вызывать программы: cl, lib, gcc, ar, путь к ним указывать **не** нужно. Заметьте, что если переименовать c.bat.txt или c.sh.txt в c.bat, то его можно будет запустить локально.

Тестирующий код жюри будет корректно вызывать эти функции, все размеры будут положительными и не будут превышать 1 000. Кроме того, гарантируется, что при вызове любой функции все параметры-указатели будет указывать на неперекрывающиеся куски

памяти. Общее количество вызовов и размеры будут достаточно малы, чтобы решение с правильной асимптотикой легко укладывалось по времени. Для вашего удоства при обнаружении ошибки в лог будет писаться причина ошибки: имя функции, и иногда даже входные данные.

Содержимое хедера myblas.h:

```
#pragma once
//====== уровень 1 ======
// (все векторы длины n)

// скопировать вектор из X в Y
void dcopy(int n, const double *X, double *Y);
// обменять местами содержимое векторов X и Y
void dswap(int n, double *X, double *Y);
// домножить вектор X на коэффициент alpha
void dscal(int n, double alpha, double *X);
// прибавить к вектору Y вектор X, умноженный на коэффициент alpha
void daxpy(int n, double alpha, const double *X, double *Y);
// вычислить скалярное произведение векторов X и Y
double ddot(int n, const double *X, const double *Y);

//====== уровень 2 ======

// вычислить вектор (alpha*A*X + beta*Y) длины m, и записать его в Y
// здесь A --- матрица размера m на n, X --- вектор длины n, а Y --- вектор длины
m
void dgemv(
    int m, int n,
    double alpha, const double *A, const double *X,
    double beta, double *Y);
// вычислить матрицу (alpha*X*Yt + A) и записать её в A
// здесь Yt --- это транспонированный вектор Y, то есть записанный как вектор-
строка
//       A --- матрица размера m на n, X --- вектор длины m, а Y --- вектор длины
n
void dger(
    int m, int n,
    double alpha, const double *X, const double *Y,
    double *A);

//====== уровень 3 ======

// вычислить матрицу (alpha*A*B + beta*C) и записать её в C
// здесь A --- матрица размера m на k, B --- матрица размера k на n,
//       C --- матрица размера m на n
void dgemm(
    int m, int n, int k,
    double alpha, const double *A, const double *B,
    double beta, double *C);
```

Задача 2. Сборка

Источник:	базовая
Имя входного файла:	--
Имя выходного файла:	--
Ограничение по времени:	--
Ограничение по памяти:	--

В этой задаче предлагается написать простой скрипт для сборки заданной программы. Программа использует библиотеки zlib и minizip, чтобы прочитать текстовый файл из zip-архива, а затем выводит гистограмму символов этого текста. Вам нужно собрать и саму программу, и эти библиотеки. Полный исходный код программы можно скачать по этой ссылке.

При сборке следует иметь ввиду следующее:

- Из zlib нужно линковать все файлы исходного кода, а из minizip — только unzip.c и ioapi.c.
- В файлах подключаются хедеры без правильного пути — вам придётся настроить include paths.
- Входной zip-файл запаролен. Чтобы включить в minizip поддержку паролей, нужно определить макрос USE_CRYPT.

На проверку требуется отправить один файл: c.bat.txt. Это скрипт сборки, который должен содержать одну или несколько команд, которые будут запускаться в командной строке. Команды будут запускаться по очереди в той же директории, куда будет распаковано содержимое вышеуказанного архива с исходным кодом (файл test.c будет в той же директории). В результате сборки должен получиться файл checked_solution в той же директории.

Разрешается вызывать программы: lib, gcc, ar, путь к ним указывать не нужно. Среди этих небогатых возможностей вы можете самостоятельно выбрать способ сборки и компилятор. Учтите, что у вас нет возможности копировать файлы, удалять файлы, и менять текущую директорию — придётся думать, как обойтись без этого. Заметьте, что если переименовать c.bat.txt в c.bat, то его можно будет запустить локально.

Задача 3. Максимальный отрезок

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	разумное
Ограничение по памяти:	разумное

Дан массив A длины N, все элементы массива Ai неотрицательные. Далее нужно ответить на M вопросов. В каждом вопросе задана позиция L в массиве и число S. Нужно найти самый длинный отрезок с левым краем в L, сумма элементов которого **не** превышает S. Вывести требуется правый край R такого отрезка.

В данной задаче края отрезка задаются так, как принято в языке C, то есть:

1. Элементы массива нумеруются начиная с нуля.
2. Отрезок с краями $L < R$ включает L-ый элемент, но не включает R-ый элемент. Если $L = R$, то отрезок пустой.

В первой строке дано два целых числа: N — количество элементов массива A и M — количество вопросов ($1 \le N,M \le 10^5$).

В следующих N строках записаны элементы массива Ai, по одному в строке. Все эти числа целые неотрицательные и не превышают 10^9.

В оставшихся M строках записаны вопросы, по одному в строке. Вопрос описывается двумя целыми числами: L — левый край искомого отрезка и S — насколько большой может быть сумма элементов отрезка ($0 \le L \le N$, $0 \le S \le 10^{15}$).

Использование хедеров в этой задаче запрещается. Решение задачи должно быть разбито на три исходных файла: sum.c, query.c и main.c Ровно эти файлы следует отправлять в систему тестирования.

В файле sum.c должны быть определены следующие функции:

```
// выполнить подготовку массива: запускается один раз перед запусками Sum
// здесь arr[0..n) -- это массив A из входного файла
void Init(const int *arr, int n);
// найти сумму элементов A на отрезке [l..r)
// использует данные, посчитанные функцией Init
int64_t Sum(int l, int r);
```

Используя эти функции, нужно реализовать в файле query.c ещё одну функцию:

```
// находит самый длинный отрезок с началом в l и суммой не более sum
// возвращает правый край искомого отрезка
int Query(int l, int64_t sum);
```

В файле main.c должна быть функция main (точка входа), чтение и запись данных, вызовы вышеописанных функций.

Можно использовать глобальные и статические переменные для хранения любых данных.

Пример

input.txt	output.txt
10 7 1 4 0 5 6 0 0 1 5 3 0 100 0 5 4 11 4 12 4 13 10 100 8 0	10 3 8 9 9 10 8

Комментарий

Можно реализовать функцию Init за время $O(N)$, так что функция Sum будет работать за время $O(1)$. Для этого надо предподсчитать суммы на $[0, k)$ для всех k, а сумму на отрезке вычислять через две такие суммы.

Заметим, что сумма элементов массива на отрезке $[l, r)$ аналогична интегралу функции на отрезке $[a, b]$. В таком случае идея данной задачи является дискретным аналогом формулы Ньютона-Лейбница =)

Задача 4. Разложение на простые

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дано несколько натуральных чисел, нужно разложить каждое число X на простые множители. Искомое разложение имеет вид:

$$X = p_1^{t_1} p_2^{t_2} \cdot \ldots \cdot p_k^{t_k}$$

При этом все числа p_i должны быть простыми и упорядоченными строго по возрастанию, а числа t_i должны быть целыми положительными.

Требуется реализовать следующую функцию для поиска разложения:

```
// структура, в которой хранится разложение на простые множители
typedef struct Factors
{
    int k;          // сколько различных простых в разложении
    int primes[32]; // различные простые в порядке возрастания
    int powers[32]; // в какие степени надо эти простые возводить
} Factors;
// функция, которая находит разложение числа X и записывает его в структуру res
void Factorize(int X, Factors *res);
```

Использование хедеров в этой задаче запрещается. Функция Factorize должна быть реализована в исходном файле factorize.c. Всё остальное (точка входа, чтение/запись и пр.) должно быть в исходном файле main.c. Отправлять нужно только эти два исходных файла.

Учтите, что файлы в вашей программе будут частично подменяться на файлы “жюри”, чтобы проверить, что вы действительно разделили программу на части как описано.

Формат входных данных

В первой строке дано целое число M — количество запросов ($1 \le M \le 1\,000$). В каждой из остальных M строк записано одно целое число X ($1 \le X \le 10^9$).

Формат выходных данных

Для каждого числа X нужно вывести его разложение на простые множители в отдельной строке. Формат вывода должен быть полностью аналогичен формату примера. Сначала пишется число X, потом знак равенства. Если простых в разложении нет ($k = 0$), то надо написать просто единицу. Иначе нужно вывести все степени в виде p^t, разделённые знаком умножения. Между соседними токенами должен стоять ровно один пробел.

Пример

input.txt	output.txt
15 1 2 3 4 5 6 7 8 9 10 1000000000 999999999 999999987 999999997 999999937	1 = 1 2 = 2^1 3 = 3^1 4 = 2^2 5 = 5^1 6 = 2^1 * 3^1 7 = 7^1 8 = 2^3 9 = 3^2 10 = 2^1 * 5^1 1000000000 = 2^9 * 5^9 999999999 = 3^4 * 37^1 * 333667^1 999999987 = 3^1 * 7^2 * 6802721^1 999999997 = 71^1 * 2251^1 * 6257^1 999999937 = 999999937^1

Задача 5. Численное дифференцирование

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дана гладкая функция $f(x)$, определённая на отрезке [0, 1], и набор точек x_i. Требуется вычислить производную $\frac{df}{dx}(x)$ в заданных точках x_i.

На каждом тесте ваша программа будет собираться заново. К списку отправленных вами исходных файлов будет добавляться файл $\hat{f}unc.c$, в котором определена одна программная функция Function. Она вычисляет значение $f(x)$ по заданному аргументу x. Например, так выглядит содержимое $\hat{f}unc.c$ на первом тесте:

```
#include <stdlib .h>
double Function(double x)
{
    if (x < 0.0 || x > 1.0)
        exit(666); // out of range
    return x * x - x;
}
```

Остальные входные данные (точки x_i) задаются в файле input.txt.

Формат входных данных

В первой строке дано целое число M — количество точек ($1 \le M \le 1\,000$). В каждой из остальных M строк записано одно вещественное число xi ($0 \le x_i \le 1$).

Формат выходных данных

Для каждого числа x_i нужно вывести значение производной $f'(x_i)$. Значение считается верным, если его абсолютная или относительная погрешность не превышает $3 \cdot 10^{-6}$. Тест засчитывается, если все вычисленные значения верные.

Рекомендуется выводить все значения с максимальной точностью.).

Пример

input.txt	output.txt
6 0.0 0.2 0.4 0.6 0.8 1.0	-1.000000000000000 -0.600000000000000 -0.200000000000000 0.200000000000000 0.600000000000000 1.000000000000000

Комментарий

Для решения задачи вспомните определение производной.

Задача 6. Количество простых

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В этой задаче нужно опять написать код для поиска простых чисел. Требуется отправить на проверку два файла: primes.c и main.c. Файл primes.c должен включать (#include) в себя файл primes.h и реализовывать объявленные в нём три функции. Файл main.c должен содержать функцию main, в которой тестируются объявленные в primes.h функции с использованием assert. Сам файл primes.h отправлять не обязательно, он будет автоматически добавлен при проверке (изменять его содержимое вы не можете).
Содержимое хедера primes.h таково:

```
#ifndef PRIMES_9183746069462
#define PRIMES_9183746069462
// returns: 1 if x is prime number, 0 otherwise
int isPrime(int x);
// returns minimal prime number p such that p >= x
int findNextPrime(int x);
// returns the number of primes x such that l <= x < r
int getPrimesCount(int l, int r);
#endif
```

Гарантируется, что числа x, l, r лежат в диапазоне от 0 до 10^7 включительно. Кроме того, при вызове getPrimesCount выполняется $l \leq r$. Гарантируется, что за запуск программы тестирующий код делает не более $2 \cdot 10^6$ вызовов функций. Эти условия соблюдает тестирующий код жюри, и ваш тестирующий код тоже должен их соблюдать.
При проверке будет добавлен хедер primes.h. Кроме того, main.c и primes.c будут поочерёдно подменяться на файлы жюри, чтобы проверить правильность работы функций и качество тестирования.
Внимание: Единица трансляции primes.c **не** должна определять никаких публичных символов, кроме описанных трёх функций. Если вам нужно завести ещё какую-то функцию или глобальную переменную, сделайте её приватной для единицы трансляции.

Комментарий

Следует использовать решето Эратосфена для поиска простых чисел. Известно, что среди чисел порядка N примерно каждое $(\ln N)$-ое является простым — об этом гласит Prime Number Theorem. Кроме того, расстояние между соседними простыми называется prime gap и растёт примерно логарифмически.

Задача 7. Indirection

Источник:	повышенной сложности
Имя входного файла:	---
Имя выходного файла:	---
Ограничение по времени:	5 секунд
Ограничение по памяти:	разумное

С учётом использования динамических библиотек получается, что вызов функции не всегда выполняется напрямую. Часто выполняется несколько косвенных переходов разного вида перед тем, как процессор попадает на тело функции. В этой задаче предлагается научиться распознавать несколько видов таких косвенных переходов на архитектуре x86.

Первый тип — это безусловный переход в заданное место по относительному адресу:

000CEEEA E9 AD 35 00 00 jmp _GetProcAddress@8 (0D249Ch)

В данном случае первый байт E9 задаёт opcode команды, а остальные четыре байта — относительный адрес пункта назначения в little-endian. Относительный адрес (000035AD) получается вычитанием из абсолютного адреса пункта назначения (000D249C) адреса конца самой команды (000CEEEA + 5 байт). Относительный адрес является знаковым 32-битным целым, в частности, он может быть отрицательным.

Второй тип — это безусловный переход по абсолютному адресу, который записан в заданной ячейке памяти. Адрес ячейки задаётся абсолютный (важно: 32-битный режим):

000D249C FF 25 00 50 19 00 jmp dword ptr [__imp__GetProcAddress@8 (0195000h)]

Здесь первые два байта FF 25 кодируют opcode команды, а остальные четыре байта — откуда брать адрес пункта назначения. В данном случае из памяти по адресу 00195000 читается четырёхбайтовый адрес пункта назначения, и выполняется переход на него.

От вас требуется создать динамическую библиотеку с одной экспортированной функцией resolve. Сигнатура функции:

```
void* resolve(void *address);
```

В функцию передаётся абсолютный адрес в памяти, на который передаётся управление. Она должна вернуть адрес, в котором будет управление, когда закончатся переходы описанных выше типов. Если выполняемые косвенные переходы зацикливаются, то нужно вернуть

NULL = 0.

Точное поведение функции должно быть таким:

1. Прочитать два байта по заданному адресу address.
2. Если первый байт имеет значение E9, то прочитать оставшиеся байты перехода первого типа и вычислить адрес destAddr, на который он ведёт.

3. Если прочитанные байты имеют значения FF и 25, то прочитать ещё 4 байта перехода второго типа и загрузить адрес destAddr, на который он ведёт.
4. Если распознать переход не удалось, то вернуть address из функции.
5. Если распознать переход удалось, то вернуть resolve(address) (рекурсивный вызов).

Кроме того, от вас требуется реализовать защиту от бесконечной рекурсии: если такая ситуация обнаружена, нужно вернуть NULL.

Нужно отправить в систему тестирования ваш исходный код, а также скрипт c.bat.txt, который собирает из исходного кода динамическую библиотеку resolve.dll при помощи Visual C 32-bit. В системе тестирования код жюри будет импортировать вашу функцию и вызывать её для различных адресов. Убедитесь, что библиотека импорта resolve.lib также создаётся.

К сожалению, эта задача имеет смысл только в 32-битном режиме, из-за этого тестирования под Linux нет.

Гарантируется, что:

1. При правильном распознавании переходов ваш код сможет прочитать все нужные байты из памяти.
2. Общее количество выполненных косвенных переходов на протяжении всего тестирования не превысит $3 \cdot 10^5$; при условии, что ваша функция не будет выполнять никакой косвенный переход дважды в течение одного вызова resolve.

Состояние памяти может изменяться между вызовами функции resolve, но **не** изменяется в процессе работы resolve.

Пример использования функции resolve можно скачать здесь.